

Ten Queries

DB25's SQL front end, measured against PostgreSQL 18 and DuckDB 1.5 at the same boundary

September 2026 · DB25 project · [space-rf-org/db25](#)

A disclaimer, because it should come first

We do not chase numbers. This one is for fun.

Query-planner benchmarks are the easiest numbers in databases to make say whatever you want. Choose the fixture, choose the queries, choose where to start and stop the clock, and any system can be made to win. Nobody should make a decision from what follows. It is ten queries against a 192,000-row fixture on one 4-core VM. It is not a workload, it is not TPC anything, and DB25 does not yet have an executor, so it cannot run a workload even if we had one.

So why publish it?

Because a query planner is one of the few parts of a database whose quality can be inspected without running anything. You can read what it chose and what it believed, line by line, and hold both against systems that have been getting it right for thirty years. That comparison stays meaningful even where the timings do not, and it is worth doing in public.

We also publish the numbers that make us look bad. There are several, they are all in one place, and section 7 says what they are.

1. The boundary, and why it is the honest one

DB25 is a SQL front end: tokenizer → parser → semantic analyzer → binder → logical plan → optimizer → **physical plan**. There is no execution engine. So the only comparison that is not a lie is one where PostgreSQL and DuckDB are stopped at exactly the same place: SQL text in, chosen physical plan out, executor never started.

Both of them can be stopped there:

system	how it is stopped	what is included
DB25	<code>lower()</code> returns the physical plan	tokenize, parse, analyze, bind, logical optimize, Cascades search
PostgreSQL 18.6	<code>EXPLAIN</code> without <code>ANALYZE</code>	parse, rewrite, plan (two numbers — see below)
DuckDB 1.5.5	in-process <code>EXPLAIN</code>	parse, bind, optimize, physical plan

PostgreSQL is reported two ways because its own `Planning Time` **excludes raw parse and rewrite**, while DB25's number includes parse. Reporting only `Planning Time` would flatter us; reporting only wall time would

flatter PostgreSQL by loading it with protocol overhead. Both are in the table. Where they disagree, believe the one that is worse for us.

This boundary is not neutral and we are not pretending it is. It favours DB25 in one way — everything is in-process, with no client protocol and no catalog round trip — and favours the others in another — they have real statistics, which DB25 does not, and statistics cost time to consult. Section 8 lists the rest.

Machine and versions

4 × Intel Xeon @ 2.80 GHz, 15 GB RAM, Linux 6.18, gcc/g++ 13.3, everything at -O2, all in one session, with identical fixtures and ANALYZE run on both engines.

- **DB25** at 0c99641, Release build.
- **PostgreSQL 18.6**, built from source at REL_18_STABLE with -O2, --without-icu, C locale. Stated because it matters: this is not a distribution build, and a distribution build with ICU, readline and a UTF-8 collation will not produce identical microseconds. The estimates it produces are unaffected by any of that; the timings are its own build's.
- **DuckDB 1.5.5**, the released Python wheel, in-process.

Fixture

table	rows	shape
items	100,000	fact; FKs to orders, prod
orders	50,000	FK to cust
emp	20,000	FK to dept
cust	20,000	
prod	2,000	
dept	50	

Every join predicate in every query is a foreign key onto a primary key. That is deliberate: it is the case where the right answer is *knowable*, so a wrong estimate has nowhere to hide.

2. The ten queries, verbatim

```
-- q1  filter
SELECT name FROM emp WHERE salary > 1000;

-- q2  two-way join
SELECT e.name, d.name FROM emp e JOIN dept d ON e.dept_id = d.id;

-- q3  four-way join
SELECT c.name FROM cust c
      JOIN orders o ON c.id = o.user_id
      JOIN items  i ON o.id = i.order_id
      JOIN prod   p ON i.prod_id = p.id;

-- q4  six-way join
SELECT c.name FROM cust c
      JOIN orders o ON c.id = o.user_id
      JOIN items  i ON o.id = i.order_id
      JOIN prod   p ON i.prod_id = p.id
      JOIN emp    e ON e.id = c.id
      JOIN dept   d ON d.id = e.dept_id;

-- q5  group by + aggregates
SELECT dept_id, COUNT(*), SUM(salary) FROM emp GROUP BY dept_id;

-- q6  having
SELECT dept_id, SUM(salary) FROM emp GROUP BY dept_id HAVING SUM(salary) > 100;

-- q7  window function
SELECT name, RANK() OVER (PARTITION BY dept_id ORDER BY salary DESC) FROM emp;

-- q8  correlated EXISTS
SELECT e.name FROM emp e
      WHERE EXISTS (SELECT 1 FROM orders o WHERE o.user_id = e.id);

-- q9  CTE + join + non-equi condition
WITH s AS (SELECT dept_id, AVG(salary) AS a FROM emp GROUP BY dept_id)
SELECT e.name FROM emp e JOIN s ON e.dept_id = s.dept_id
      WHERE e.salary > s.a;
```

And the one we care most about, because it is the one that exercises everything at once — CTE, aggregate, window function, inner join, `LEFT JOIN LATERAL` with a correlated scalar aggregate, a non-equi filter, an `ORDER BY` over a window output, and a `LIMIT` :

```
-- q10 reference query
WITH dept_stats AS (
  SELECT dept_id, AVG(salary) AS avg_sal, COUNT(*) AS headcount
  FROM emp GROUP BY dept_id
)
SELECT e.name,
       e.salary,
       ds.avg_sal,
       RANK() OVER (PARTITION BY e.dept_id ORDER BY e.salary DESC) AS rnk,
       ord.total
FROM emp e
JOIN dept_stats ds ON e.dept_id = ds.dept_id
LEFT JOIN LATERAL (
  SELECT SUM(o.total) AS total FROM orders o WHERE o.user_id = e.id
) ord ON true
WHERE e.salary > ds.avg_sal
ORDER BY rnk
LIMIT 10;
```

`LATERAL` is the interesting part. Its right side references `e.id` from the left, so it cannot be evaluated standalone: the only admissible implementation is a nested loop that re-evaluates the subquery per left row. DB25's applicability rules say so structurally rather than discovering it by costing — a hash join there would not merely be slower, it would not be executable. The plan in Appendix B shows it choosing `NestedLoopJoin kind=leftlateral` with an `(outer d1 #0)` correlation reference, which is the front end getting a genuinely hard construct right.

3. Planning time

Median of 500 runs (DB25) / 200–300 runs (others), warm, μ s per query.

	query	DB25	PG 18 parse+plan	PG 18 planner only	DuckDB
q1	filter	7.04	13.41	5.00	113.0
q2	2-way join	12.33	38.71	17.00	306.7
q3	4-way join	48.73	98.93	49.00	877.3
q4	6-way join	74.02	242.63	161.00	1641.3
q5	group by	11.62	24.62	9.00	299.8
q6	having	13.98	30.61	11.00	379.3
q7	window	10.37	24.75	9.00	225.9
q8	EXISTS	12.92	51.01	23.00	367.9
q9	CTE + join	25.70	76.04	31.00	695.5
q10	reference	67.63	169.74*	67.50	1518.1

* q10's wall figure still carries some inflation: the baseline subtracts a one-line `EXPLAIN`, and q10's plan output is thirty lines, so part of that number is psql formatting rather than planning. Its planner-only figure is clean.

Re-running the wall measurement at N=400 instead of N=200 moved every figure by under 8% and none of the orderings, so these are not one-sample numbers.

Read against the column that is worst for us — PostgreSQL planner-only, which excludes the parse and rewrite that DB25's number includes — DB25 wins the four queries with real planning in them: q4 (74.0 vs 161), q8 (12.9 vs 23), q9 (25.7 vs 31), q2 (12.3 vs 17). It ties q3 (48.7 vs 49.0) and q10 (67.6 vs 67.5), and loses q1, q5, q6 and q7 — the four where PostgreSQL's planner has almost nothing to do and DB25 is still paying for a tokenizer and a memo. Read against parse+plan, which is the like-for-like boundary, DB25 wins all ten.

DuckDB is 16–29× slower than DB25 throughout, and roughly an order of magnitude slower than PostgreSQL's planner. That is not a defect in DuckDB. It is an analytical engine whose planning cost is amortised over multi-second scans, and it spends the time on things — a sampling-based join-order optimiser, runtime re-optimisation hooks — that only pay off when there is an executor to pay them back. It is in the table because it is a fair third data point at this boundary, not because 1.6 ms is a problem for anyone running DuckDB.

The honest framing of this whole table: planning time is not why DB25 exists, and at execution scale it is noise. What planning time is good for is what it says about the *shape* of the search — DB25 explores 63 physical candidates across 31 optimization goals for q4, prunes 25 of them by branch-and-bound, and finishes in 74 μs. That is the number worth caring about, and it is in section 5.

4. Cardinality estimates — the numbers that actually matter

A planner's speed is a convenience. Its **estimates** are what decide the plan, and a plan chosen from a wrong estimate is slow no matter how fast it was chosen.

Top-node estimated rows against the true result size:

	actual	DB25	PostgreSQL 18	DuckDB
q1 filter	18,999	2,000	18,999	4,000
q2 2-way join	19,600	20,000	20,000	22,222
q3 4-way join	99,950	100,000	99,934	26,961
q4 6-way join	98,000	100,000	99,934	15,695
q5 group by	50	2,000	50	45
q6 having	50	200	17	9
q7 window	20,000	20,000	20,000	20,000
q8 EXISTS	19,999	2,000	19,690	4,000
q9 CTE + join	10,000	2,000	6,667	663
q10 reference	10	10	10	5,000

Aggregated as q-error — the geometric mean of $\max(\text{est}/\text{act}, \text{act}/\text{est})$, the standard measure since Leis et al.'s *How Good Are Query Optimizers, Really?*:

system	geometric mean q-error	statistics available
PostgreSQL 18	1.17×	full: histograms, n_distinct, MCVs, correlation
DB25 (today)	3.09×	none
DuckDB 1.5	5.58×	full, with <code>ANALYZE</code> run

DB25 sits between the two mature systems while consulting **no column statistics at all** — and we want to be careful about what that does and does not mean. It does not mean DB25 estimates better than DuckDB. DuckDB deliberately leans on runtime adaptivity rather than static estimates: it samples and re-orders during execution, so a rough `EXPLAIN` cardinality costs it much less than the same error would cost a system that commits to a plan up front. DB25 has no executor to adapt with, so a static estimate is all it will ever have — which is exactly why these numbers get taken seriously here.

5. Search shape

What DB25 is actually doing, per query:

	memo groups	candidates	goals	pruned
q1 filter	3	3	3	0
q2 2-way join	4	7	5	1
q3 4-way join	11	45	23	18
q4 6-way join	16	63	31	25
q8 EXISTS	4	5	4	1
q9 CTE + join	7	11	10	1
q10 reference	15	20	23	3

Goals exceeding groups is the point of the whole design. A goal is a (group, required-properties) pair: the same logical subtree optimized more than once, for different demands from above. Bottom-up planners cannot ask that question — they settle the join before the `ORDER BY` is considered and then sort whatever came out. q4 optimizes 16 groups for 31 goals and q10 optimizes 15 for 23, which is the top-down property-directed search visibly doing what it exists to do. q1, q5 and q7 have goals equal to groups, which is also correct: nothing in them demands a property from below.

Pruning at q4 discards 25 of 63 candidates by branch-and-bound before they are fully costed. That is where the 74 μ s comes from: not from being clever per candidate, but from not costing the ones that already lost.

6. Dissection: q9, line by line, across three planners

```
WITH s AS (SELECT dept_id, AVG(salary) AS a FROM emp GROUP BY dept_id)
SELECT e.name FROM emp e JOIN s ON e.dept_id = s.dept_id
WHERE e.salary > s.a;
```

Moderately complex on purpose: it has a CTE that must be inlined, an aggregate over 20,000 rows into 50 groups, a self-join of `emp` against that aggregate, an equi-key (`dept_id`), and a **non-equi residual** (`salary > a`) that cannot be a hash key. Actual result: **10,000 rows**.

What each system produced

PostgreSQL 18 (76.0 µs parse+plan, 31 µs planner only):

```
Hash Join (rows=6667)
  Hash Cond: (e.dept_id = emp.dept_id)
  Join Filter: ((e.salary)::numeric > (avg(emp.salary)))
  -> Seq Scan on emp e (rows=20000)
  -> Hash (rows=50)
      -> HashAggregate (rows=50) Group Key: emp.dept_id
          -> Seq Scan on emp (rows=20000)
```

DuckDB (695.5 µs):

```
PROJECTION (~663 rows)
├─ HASH_JOIN INNER
│   Conditions: dept_id = dept_id
│               CAST(salary AS DOUBLE) > a
│   ├─ SEQ_SCAN emp (~20,000)
│   └─ PERFECT_HASH_GROUP_BY Groups: dept_id Aggregates: avg(salary) (~45)
│       └─ SEQ_SCAN emp (~20,000)
```

DB25 (25.7 µs):

```
(Project exprs=[(col #0)]
 (Filter pred=(> (col #2) (col #4))
  (HashJoin kind=inner build=right keys=[(L#1 R#0)]
   (SeqScan table=emp fmt=row)
   (Project exprs=[(col #0) (col #1)]
    (HashAggregate keys=[(col #0)] aggs=[(AVG (col #1))]
     (SeqScan table=emp fmt=row))))))
```

Where they agree

All three make the same four decisions, and they are the decisions that matter:

1. **Inline the CTE.** None of the three materialises `s`. PostgreSQL 12+ inlines a non-recursive CTE referenced once; DuckDB and DB25 never materialise it in the first place. A materialised CTE here would cost a temporary relation for 50 rows.
2. **Hash-aggregate, not sort-aggregate.** Nothing downstream wants `dept_id`-order, so paying $n \log n$ to get grouping for free loses. DB25 reaches this by costing both: `StreamingAggregate` is a candidate in the same memo group, priced at `streaming_aggregate_row` (0.5) against `hash_aggregate_row` (1.1) — cheaper per row — and loses anyway, because it would have to enforce a Sort of 20,000 rows to become applicable and the search charges it for the enforcer it causes.
3. **Hash join, building the 50-row side.** All three build the small side. In DB25 this is not a rule — `build_right` is a *costed candidate*, both orientations are enumerated, and the comparison $50 \times \text{hash_build_row} + 20000$

$\times \text{hash_probe_row}$ against $20000 \times \text{hash_build_row} + 50 \times \text{hash_probe_row}$ settles it. Both orientations are among q9's 11 candidates, and one of them loses.

4. `salary > a` is not a join key. All three recognise that an inequality cannot be hashed and re-check it per candidate row.

That four-way agreement is the actual headline of this section. A young planner agreeing with PostgreSQL and DuckDB on every structural decision of a query with a CTE, an aggregate, a join and a residual is the thing worth reporting — far more than any microsecond count.

Where they differ, and why

Placement of the residual. PostgreSQL folds `salary > a` into the join as a `Join Filter`. DuckDB folds it in as a second join condition. DB25 emits a separate `Filter` node above the join.

Semantically identical; the difference is one operator boundary. Folded in, the predicate is evaluated inside the probe loop on each matched pair before the row is materialised. Hoisted out, the row is produced and then discarded. For an inner join with a 50-row build side this is a small constant either way, and DB25's cost model prices the Filter honestly (`filter_row` per input row) so it is not free in the comparison — but it is a genuine difference, and the folded-in form is the better one. It is a known simplification of DB25's join lowering, not an oversight: the `residual` conjuncts *are* carried on the join node (`residual=[...]` in the IR, see q10's lateral join in Appendix B), so the machinery exists; the inner-join path currently prefers to hoist. That is a lowering change, not a model change.

Estimates. Here the three diverge sharply, and the reason is instructive:

	estimate	actual	implied selectivity for <code>salary > a</code>
actual	—	10,000	0.50
PostgreSQL	6,667		0.333
DB25	2,000		0.100
DuckDB	663		0.033

Every system gets the join right (20,000 rows: 20,000 employees each matching exactly one department average) and then applies a **guess** to the inequality. PostgreSQL's guess is `1/3`, the Selinger default for an inequality against an unknown value. DB25's is its flat `filter_selectivity = 0.1`. DuckDB's works out around `0.033`.

PostgreSQL wins this one by having the largest guess, not the smartest one. *Nobody* has a statistic for "compare a column against a per-group aggregate of that same column" — it needs a correlation between `salary` and its own group mean, which no histogram carries. The true answer, 0.5, is what you would get from knowing the distribution is roughly symmetric within each department, and that is an assumption none of the three is entitled to make.

That is the honest lesson of the dissection: DB25's join estimate is as good as PostgreSQL's, and its *filter* estimate is a flat constant with nothing behind it. Which is section 7.

7. Where DB25 is worst, stated plainly

Look again at the cardinality table. Every DB25 miss is in one place.

query	what it got wrong	the constant responsible
q1	2,000 vs 18,999	<code>filter_selectivity = 0.1</code>
q8	2,000 vs 19,999	<code>join_selectivity = 0.1</code> , via the semi-join rule
q9	2,000 vs 10,000	<code>filter_selectivity = 0.1</code>
q5	2,000 groups vs 50	<code>group_selectivity = 0.1</code>
q6	200 vs 50	<code>group_selectivity = 0.1</code>

Every join estimate is within 3% of the truth. Every *selectivity* estimate is a flat constant — including the semi-join's, which is the one join-shaped rule that still carries a bare guess, because how many left rows have a match is a fact about the data rather than a fact about the join.

Those constants are honest: they are named, they live in one place, and their comments say they are guesses rather than derivations. But a guess is what they are, and `0.1` for `WHERE salary > 1000` on a column whose values run 0–20,000 is a guess that is wrong by 9.5×.

No better rule fixes this. A filter's selectivity is a property of the data, and estimating it needs what PostgreSQL has: per-column histograms, distinct counts, and most-common-value lists from the catalog. That is the next piece of work, it is bounded, and it is the reason DB25's 3.09× does not become 1.2× by being clever.

The same applies to `group_selectivity`. There are exactly 50 departments; `n_distinct(dept_id)` would say so. DB25 guesses 10% of input rows and is 40× out.

Also worth stating: q9 showed DB25 hoisting a residual out of a join where both mature systems fold it in. That is a real, if small, plan-quality gap, and it is recorded rather than buried.

8. What this does not show

- **No execution.** Everything here is about choosing a plan, nothing about running one. A planner that chooses well and an engine that executes well are different problems and only one of them is measured.
- **One fixture, ten queries, one machine.** Six tables and 192,000 rows. Every join is a clean foreign key. Real schemas have composite keys, skew, correlation, NULLs and outer joins in combinations this fixture does not contain — and all of those are where estimates go wrong.
- **In-process versus not.** DB25 pays no client protocol and no catalog round trip. Part of its margin is architectural, not algorithmic.
- **No statistics maintenance cost.** PostgreSQL and DuckDB paid for `ANALYZE`. DB25 has nothing to maintain because it has nothing to consult, and section 7 is the bill for that.
- **DuckDB is being measured outside its design point.** Its planning cost is built to be amortised over an executor's seconds. Reading its 1.6 ms as a weakness would be exactly the kind of meaningless number this paper opened by disclaiming.

- **These numbers move.** They were taken on 2026-09-20 against `db25@0c99641`. They will be wrong by the next increment, and that is the intended failure mode — the numbers exist to be falsified, not framed.

9. Conclusion

Two things came out of an afternoon of measurement.

One thing confirmed. DB25's search machinery is sound. Where it has the information it needs, its output matches two mature planners decision for decision: same join algorithms, same build sides, same aggregate strategies, same CTE inlining, the same refusal to hash an inequality. It does that in 74 μ s on the hardest join in the set, exploring 63 candidates and pruning 25, against PostgreSQL's 161 μ s.

One bill, itemised. Everything DB25 still gets wrong is a flat selectivity constant standing in for a statistic it does not have. Every join estimate is within 3%; every selectivity estimate is a guess. That is a known, bounded piece of work, and section 7 names it exactly.

Neither of which is a benchmark. It was fun, and it cost an afternoon.

Appendix A — reproducing

Everything is in `docs/bench/`:

file	what it is
<code>schema.sql</code> <code>data.sql</code>	the six tables and their 192,000 rows
<code>queries.txt</code>	the ten queries, <code>id label sql</code>
<code>db25_bench.cpp</code>	DB25 timing: median of N warm iterations, catalog construction hoisted out of the loop
<code>db25_plans.cpp</code>	DB25 plan shape, estimated rows, memo groups, candidates, goals, pruned
<code>pg_wall.py</code>	PostgreSQL both ways: <code>EXPLAIN (SUMMARY ON) xN</code> for planner time, and wall time for N <code>EXPLAIN</code> s minus N baseline statements for parse+plan
<code>others_bench.py</code>	PostgreSQL planner time and DuckDB in-process <code>EXPLAIN</code> , minus the same baseline

```
# Where the cluster is. The two PostgreSQL harnesses read these, so the same
# scripts point at any build without being edited.
export PSQL_BIN=/opt/pg18/bin/psql PGHOST=/tmp/pgs18 PGPORT=55433

$PSQL_BIN -h $PGHOST -p $PGPORT -U postgres -d bench -f docs/bench/schema.sql
$PSQL_BIN -h $PGHOST -p $PGPORT -U postgres -d bench -f docs/bench/data.sql

./db25_bench docs/bench/queries.txt 500      # DB25 timing
./db25_plans docs/bench/queries.txt          # DB25 plans and estimates
python3 docs/bench/pg_wall.py 200            # PostgreSQL, both boundaries
python3 docs/bench/others_bench.py 300       # PostgreSQL planner + DuckDB
```

The DuckDB fixture is built in-process by `others_bench.py` from the same row generators, so the two engines see identical data rather than merely similar data.

`pg_wall.py` builds one `psql -c` body holding N copies of the query, which is why N is 200 rather than larger: at N=400 the reference query overflows the argument list. The nine shorter queries were re-run at 400 as a stability check.

Appendix B — DB25's plan for q10

The reference query, showing the lateral join, the correlation reference, the window and the enforced sorts:

```
(Limit limit=10
 (Sort by=[#3 asc]
  (Project exprs=[(col #1) (col #3) (col #5) (col #8) (col #7)]
   (Window fns=[(RANK :over [:partition [(col #2)] :order [(col #3) desc]])]
    (Sort by=[#2 asc #3 desc]
     (Filter pred=(> (col #3) (col #5))
      (NestedLoopJoin kind=leftlateral keys=[] residual=[(lit true)]
       (Exchange to=single
        (HashJoin kind=inner build=right keys=[(L#2 R#0)]
         (SeqScan table=emp fmt=row)
         (Project exprs=[(col #0) (col #1) (col #2)]
          (HashAggregate keys=[(col #0)]
           aggs=[(AVG (col #1)) (COUNT)]
           (SeqScan table=emp fmt=row))))))
      (Project exprs=[(col #0)]
       (StreamingAggregate keys=[] aggs=[(SUM (col #1))]
        (Filter pred=(= (col #0) (outer d1 #0))
         (SeqScan table=orders fmt=row))))))))))
```

Four things to notice. The lateral is a `NestedLoopJoin` because it *must* be — its right side carries an `(outer d1 #0)` reference to the left row, so no build-once algorithm is applicable. The scalar `SUM` is a `StreamingAggregate` with no keys, which is exactly one row out regardless of input. The two `Sort` s are enforcers placed by the search, not written by the query: one to give the window its partition-and-order requirement, one for the `ORDER BY` `rnk` that the window's own output does not provide. And the root `Limit` clamps the estimate to 10 — the only query in the set where DB25, PostgreSQL and DuckDB were asked for a number that the SQL itself determines, and the only one DuckDB missed (5,000).

DB25 is an experiment in building a SQL front end where every design decision is falsifiable. The gap register (docs/gap-register.md) records what is known to be wrong, including everything in section 7.